

سیب‌زمینی

- محدودیت زمان: 10 دقیقه
- محیط: یک (16 GB VRAM) GPU، بدون اینترنت
- حجم راحل: `solution.ipynb` ≤ 1 MB
- فضای ذخیره‌سازی: 5 GB

مسئله

دوستان پیشنهاد می‌کند یک بازی حدس‌زدنی (guessing game) انجام دهید. او، در نقش داور (judge)، یک کلمه پنهان را از واژگانی ثابت انتخاب می‌کند و شما باید آن را در حداکثر 30 نوبت پیدا کنید. در هر نوبت، داور دو کلمه را مقایسه می‌کند و اعلام می‌کند کدامیک از نظر معنایی به کلمه پنهان نزدیکتر است. هر بازی با جفت ثابت lamp vs potato آغاز می‌شود، زیرا آن‌ها دو مورد از چیزهای موردعلاقه دوستان هستند. سپس برنامه شما یک کلمه جدید پیشنهاد می‌دهد. برنده مقایسه نگه داشته می‌شود و با پیشنهاد بعدی شما مقایسه می‌شود. به محض اینکه دقیقاً کلمه پنهان را پیشنهاد دهید، بازی را می‌برید. تطبیق به بزرگی و کوچکی حروف حساس نیست. هر کلمه‌ای که پیشنهاد می‌دهید باید در `dataset/vocabulary.json` باشد.

یک مثال کامل همراه با پروتکل و بارگذاری داده‌ها در `solution.ipynb` وجود دارد. می‌توانید کلاس `PublicEmbeddingPlayer` را تغییر دهید. برنامه شما یکبار مقداردهی اولیه (initialize) می‌شود و همه بازی‌ها را در یک اجرا انجام می‌دهد؛ پروتکل در آغاز هر بازی یک `PublicEmbeddingPlayer` جدید ایجاد می‌کند.

داور (Judge)

برنامه شما یک شیء JSON برای داور ارسال می‌کند و داور با یک شیء JSON پاسخ می‌دهد. در اینجا verdict یعنی «حکم».

یک مثال کامل، که در آن کلمه پنهان فقط برای توضیح پروتکل نمایش داده شده است (اگر جداول فارسی به درستی نمایش داده نشده‌اند به نسخه انگلیسی مراجعه فرمایید):

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
    <- matches: game won
-> {"status": "win"}
```

نوبت‌ها از 1 تا 30 شمارگذاری می‌شوند.

گزینه‌های verdict عبارت‌اند از first به این معنا که word1 نزدیکتر است، second به این معنا که word2 نزدیکتر است، یا same به این معنا که هر دو کلمه به یک اندازه به کلمه پنهان نزدیک‌اند.

winner_word کلمه‌ای است که برای مقایسه بعدی نگه داشته می‌شود. در صورت حکم same، کلمه نخست باقی می‌ماند.

مجموعه داده

مشترک میان همه تقسیم‌ها (split):

- `dataset/vocabulary.json` — شامل 1602 کلمه منحصر به فرد با حروف کوچک. کلمه پنهان همیشه یکی از این‌هاست.
- `dataset/public_embeddings.npy` — float32، با شکل (1602, 2560). سطر i متناظر با کلمه i در واژگان است. این‌ها embeddingهای عمومی هستند؛ داور از بازنمایی (representation) خصوصی و متفاوتی استفاده می‌کند.

تقسیم‌ها (split)، مجموعه‌هایی از کلمات پنهان هستند (اگر جداول گنگ هستند، به نسخه انگلیسی مراجعه فرمایید):

تقسیم	کلمات	پاسخ‌ها	کاربرد
test_public	120	<code>dataset/test_public.json</code>	اجرای راحل و محاسبه امتیاز توسط خودتان
test_leaderboard_a	120	پنهان	جدول رتبه‌بندی زنده
test_leaderboard_b	120	پنهان	رتبه‌بندی نهایی

هیچ تقسیم train وجود ندارد (به عبارت دیگر train split ای به شما داده نشده است) — هیچ چیز با استفاده از سطرها برچسب‌دار برآزش نمی‌شود.

دو مدل **embedding** از پیش‌آموزش‌دیده همراه با مسئله ارائه شده‌اند و می‌توان از آن‌ها استفاده کرد:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

هر دو باید از مسیر محلی خود بارگذاری شوند؛ یک شناسه **Hugging Face Hub** مانند "BAAI/bge-m3" باعث آغاز دانلود می‌شود و در نهایت با شکست مواجه می‌شود! زیرا دآوری آفلاین است. هر دایرکتوری شامل یک `example.py` قابل‌اجراست که فراخوانی آفلاین را نشان می‌دهد.

کتابخانه‌های موجود: `sentence-transformers`، `torch`، `numpy`. بدون اینترنت، بدون دانلود و بدون هیچ پکیج دیگری.

خروجی

هیچ چیز. این یک مسئله تعاملی است: راحل شما هیچ فایل پاسخی نمی‌نویسد؛ بلکه همان‌طور که در بالا توضیح داده شد، از طریق `stdin/stdout` با داور ارتباط برقرار می‌کند.

معیار

بازی‌ای که در نوبت t حل شود، امتیاز $\max(0, t - 10) \times 0.02 - 1.0$ می‌گیرد؛ بازی‌ای که در 30 نوبت حل نشود، امتیاز 0 می‌گیرد. بنابراین نوبت‌های 1–10 امتیاز 1.00، نوبت 20 امتیاز 0.80 و نوبت 30 امتیاز 0.60 می‌گیرند.

امتیاز مسئله شما برابر است با میانگین امتیاز بازی‌ها $\times 100$ و بین 0.00 و 100.00 خواهد بود.

محدودیت 10 دقیقه‌ای یک بودجه واحد است که راه‌اندازی، آماده‌سازی و همه 120 بازی در مجموعه آزمون را پوشش می‌دهد.

نحوه ارسال

1. `solution.ipynb` را باز کنید، `PublicEmbeddingPlayer` را ویرایش کنید و همه سلول‌ها را اجرا کنید تا مطمئن شوید درست کار می‌کند.
2. در صورت تمایل، آن را به‌صورت محلی بررسی کنید: `python local_test.py solution.ipynb --limit 5`. داور محلی از **embedding‌های عمومی** استفاده می‌کند، بنابراین امتیاز آن فقط یک راهنماست.
3. `solution.ipynb` را ذخیره کنید.
4. زبانه `Git` را در نوار کناری سمت چپ `JupyterLab` باز کنید.
5. `solution.ipynb` را **stage** کنید (آیکون **+** در کنار آن).
6. یک پیام **commit** وارد کنید و روی **Commit** کلیک کنید.
7. برای **push** کردن، روی آیکون ابر با پیکان رو به بالا کلیک کنید.
8. به این صفحه **Contest** بازگردید و روی **Submit** کلیک کنید؛ پیام **commit** باید با پیامی که ارائه کرده‌اید مطابقت داشته باشد.

دقیقاً یک فایل با نام `solution.ipynb` ارسال کنید که هرگونه آماده‌سازی و استنتاج لازم را پوشش دهد.